

DEPARTAMENTO:	Ciencias de la computacion	CARRERA:	ITIN		
ASIGNATURA:	Estructura de datos	NIVEL:	3	FECHA:	5/12/2025
DOCENTE:	Margoth Guaraca	PRÁCTICA N°:		CALIFICACIÓN:	

Implementation of binary expression trees with graphical interface in Java

Angel Steven Rodriguez Chavez

SUMMARY

In this laboratory, a complete Java application was developed for the construction, evaluation, and graphical visualization of binary trees of mathematical expressions. The system implements a Model-View-Controller (MVC) architecture that allows infix expressions to be converted into binary trees, traversal operations (inorder, preorder, and postorder) to be performed, numerical expressions to be evaluated, and the tree structure to be visualized using an interactive graphical interface. The application uses Java Swing with the Nimbus theme to provide a modern and professional visual experience. The correct functioning of the system was validated through tests with various mathematical expressions, demonstrating the program's ability to efficiently analyze, construct, and represent expression trees. The results obtained confirm that the binary tree data structure is ideal for representing and evaluating algebraic expressions, facilitating both conceptual understanding and computational processing of them.

Keywords: Binary Trees, Mathematical Expressions, Graphical Interface

1. INTRODUCTION:

Binary expression trees are a fundamental data structure in computer science, particularly in the development of compilers, interpreters, and symbolic computation systems. This laboratory practice focuses on the practical implementation of these structures, combining theoretical concepts of binary trees with object-oriented software development and graphical user interface design.

The main purpose of this laboratory is to consolidate knowledge about hierarchical data structures by building a functional application that allows the creation, manipulation, and visualization of binary expression trees. Through this practice, students develop skills in algorithm analysis, Java programming, graphical interface design, and application of the MVC architectural pattern.

The discipline and methodology employed during the development of the laboratory included systematic problem analysis, modular component design, incremental implementation of functionalities, and exhaustive validation

through test cases. This structured approach allowed us to achieve the objectives set while maintaining code quality and system functionality.

2. OBJECTIVE(S):

2.1 General Objective

- Develop a complete Java application that implements binary trees of mathematical expressions with an interactive graphical interface, applying the MVC architectural pattern.

2.2 Specific Objectives

- Implement the binary tree data structure with nodes that store operators and operands.
- Develop algorithms to construct binary trees from infix expressions.
- Implement traversal methods: preorder, inorder, and postorder.
- Design and implement a graphical user interface using Java Swing.
- Develop a graphical display component that draws the tree on screen.

3. THEORETICAL FRAMEWORK:

3.1 Binary Trees

A binary tree is a hierarchical data structure in which each node can have at most two children: a left child and a right child. Binary trees are fundamental in computer science due to their efficiency in search, insertion, and traversal operations.

3.2 Trees of Expression

An expression tree is a special type of binary tree used to represent algebraic expressions. In these trees:

- Leaf nodes contain operands (numbers or variables)
- Internal nodes contain operators (+, -, *, /, ^)
- The tree structure reflects the precedence of operators

3.3 Tree Tours

There are three main ways to traverse a binary tree:

- **Inorder (Infix):** Left → Root → Right
- **Preorder (Prefix):** Root → Left → Right
- **Postorder (Postfix):** Left → Right → Root

3.4 Analysis of Expressions

Syntactic analysis of mathematical expressions requires consideration of:

- **Operator precedence:** $^ > *, / > +, -$
- **Associativity:** Direction of evaluation for operators of the same level
- **Parentheses:** Modify the natural precedence of operators

3.5 MVC (Model-View-Controller) pattern

This is an architectural pattern that separates the application into three components:

- **Model:** Business logic and data structures
- **View:** Graphical interface and presentation
- **Controller:** Coordination between model and view

4. DESCRIPTION OF THE PROCEDURE:

4.1 System Architecture

The application follows the MVC pattern with the following structure:

Model Layer:

- **Node.java** - Binary tree node structure
- **ArbolBinario.java** - Base binary tree implementation
- **ArbolExpresion.java** - Expression-specific tree logic
- **PilaLista.java** - Stack implementation for parsing
- **NodoPila.java** - Stack node structure

View Layer:

- **ExpressionTreeView.java** - Main GUI window
- **TreePanel.java** - Custom tree visualization panel

Controller Layer:

- **ExpressionController.java** - Coordinates Model and View

Main:

- **App.java** - Application entry point

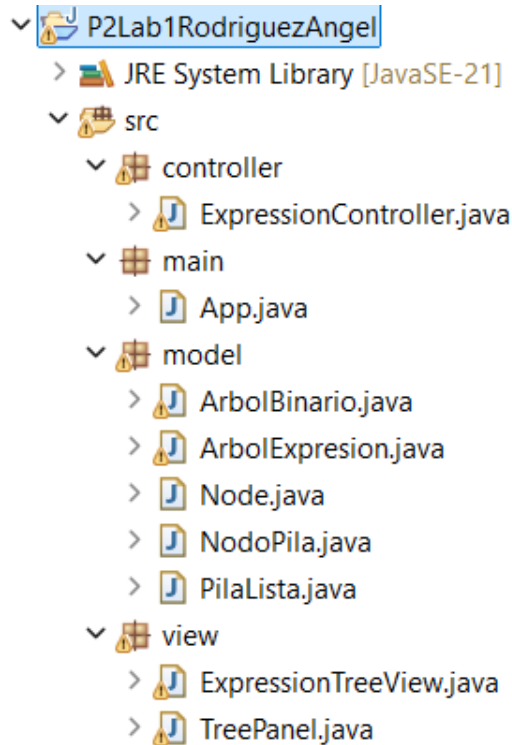


Illustration 1 Project MVC structure

4.2 Implementation Steps

Step 1: Model Layer Development

Node Class: Created a generic binary tree node with:

- Data storage for operator or operand
- Left and right child pointers
- Getter and setter methods

ArbolBinario Class: Base tree class providing:

- Root node management
- Basic tree operations
- isEmpty() verification

ArbolExpresion Class: Extended binary tree with:

- Expression parsing from infix notation
- Operator precedence handling
- Recursive evaluation algorithm
- Conversion to prefix and postfix notations

Stack Classes: Implemented stack data structure for:

- Operator management during parsing
- Parentheses matching
- Expression validation

Step 2: Expression Parsing Algorithm

Implemented a parsing algorithm that:

1. Reads the infix expression character by character
2. Uses operator precedence to determine tree structure
3. Handles parentheses for grouping
4. Builds the tree bottom-up using a stack

Step 3: Evaluation Algorithm

Recursive evaluation using postorder traversal:

1. If node is a leaf (operand), return its numeric value
2. If node is an operator:

- Recursively evaluate left subtree
- Recursively evaluate right subtree
- Apply the operator to the two results

Step 4: View Layer Development

ExpressionTreeView: Created main window with:

- Input text field for entering expressions
- "Generate" button to create the tree
- "Clear" button to reset the application
- Radio buttons for selecting traversal type
- JTabbedPane with two tabs:
 - Graphical Visualization tab (TreePanel)
 - Detailed Results tab (text output)

TreePanel: Custom JPanel with:

- Override paintComponent() for custom drawing
- Recursive tree drawing algorithm
- Node positioning calculation
- Traversal order visualization

Step 5: Tree Visualization Algorithm

Implemented recursive drawing:

```

TreePanel.java
94
95 private void drawTree(Graphics2D g2d, Node node, int x, int y, int horizontalGap) {
96     if (node == null) return;
97
98     // Draw lines to children first (so they stay behind the nodes)
99     if (node.leftSubtree() != null) {
100         int childX = x - horizontalGap;
101         int childY = y + VERTICAL_GAP;
102         g2d.setColor(LINE_COLOR);
103         g2d.setStroke(new BasicStroke(2));
104         g2d.drawLine(x, y, childX, childY);
105         drawTree(g2d, node.leftSubtree(), childX, childY, horizontalGap / 2);
106     }
107
108     if (node.rightSubtree() != null) {
109         int childX = x + horizontalGap;
110         int childY = y + VERTICAL_GAP;
111         g2d.setColor(LINE_COLOR);
112         g2d.setStroke(new BasicStroke(2));
113         g2d.drawLine(x, y, childX, childY);
114         drawTree(g2d, node.rightSubtree(), childX, childY, horizontalGap / 2);
115     }
116
117     // Draw current node
118     drawNode(g2d, node.nodeValue().toString(), x, y, node);
119 }
  
```

Illustration 2 Method for drawing the tree

Step 6: Traversal Visualization

Added interactive traversal visualization:

1. Implemented three traversal methods that return visit order
2. Added radio buttons for traversal selection
3. Modified node drawing to include order numbers
4. Order numbers displayed in orange circles on each node

Step 7: Controller Implementation

ExpressionController: Manages user interactions:

- Retrieves expression from input field
- Creates ArbolExpresion instance
- Updates both visualization and text tabs
- Handles error conditions
- Manages view updates

5. Results:

5.1 Functional Testing

Test Case 1: Simple Expression

- Input: 3+5

- Expected Tree Structure:

```

+
 / \
3  5
  
```

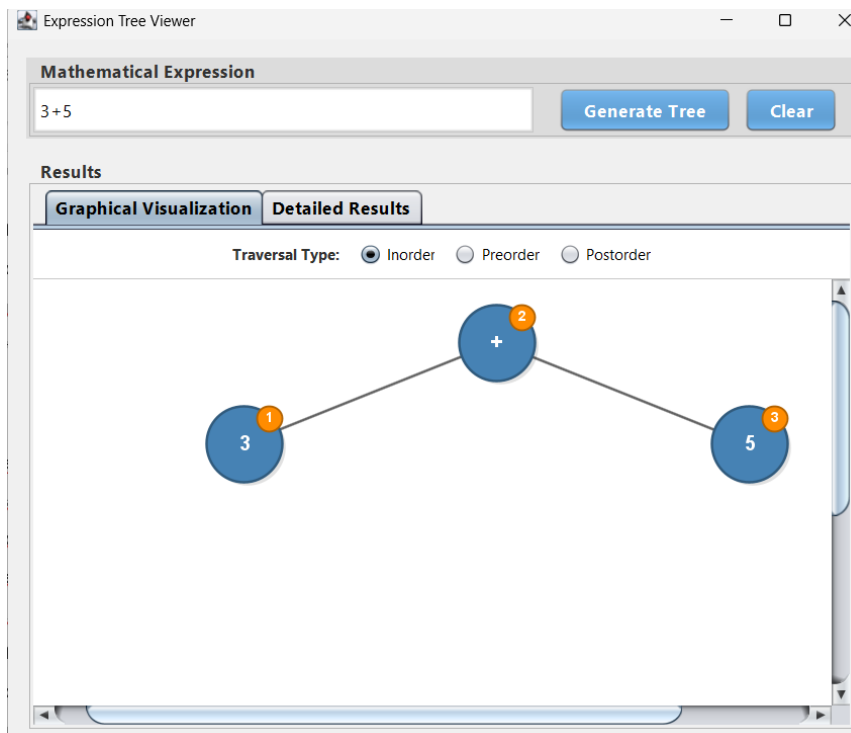
-Result: ✓ Correct visualization

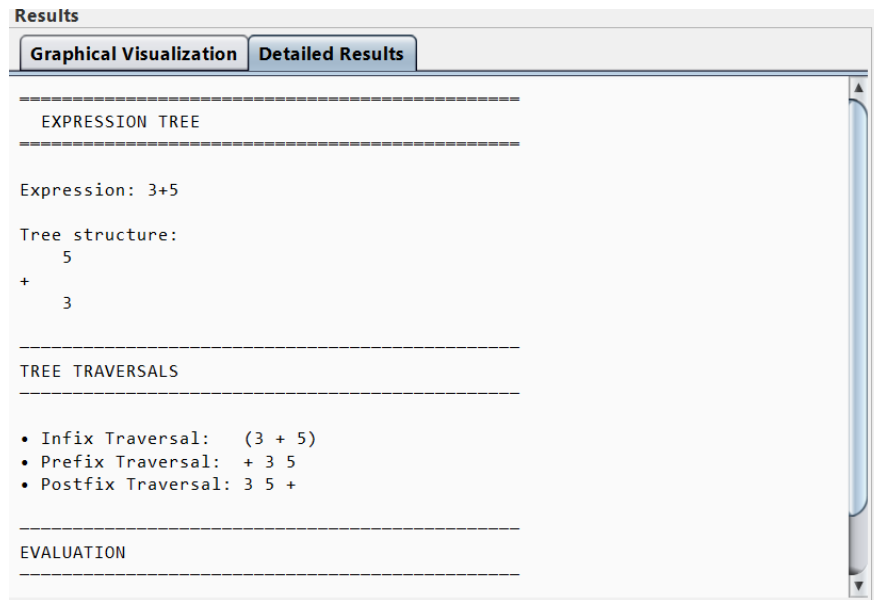
- Evaluation: 8

- Inorder: `3 + 5`

- Preorder: `+ 3 5`

- Postorder: `3 5 +`





Test Case 2: Complex Expression with Precedence

- Input: `3+5\*2`

- Expected Tree Structure:

```

  +
 / \
3   *
   / \
  5   2
  
```

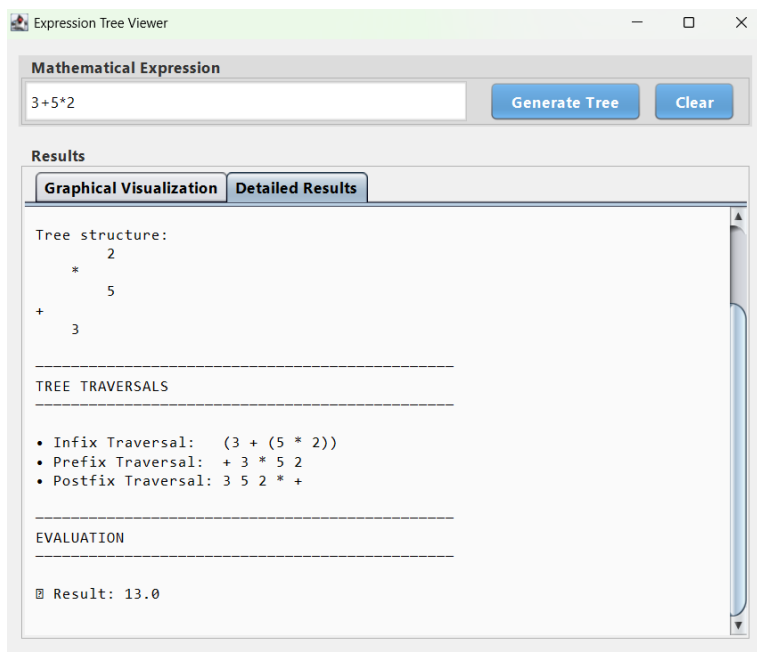
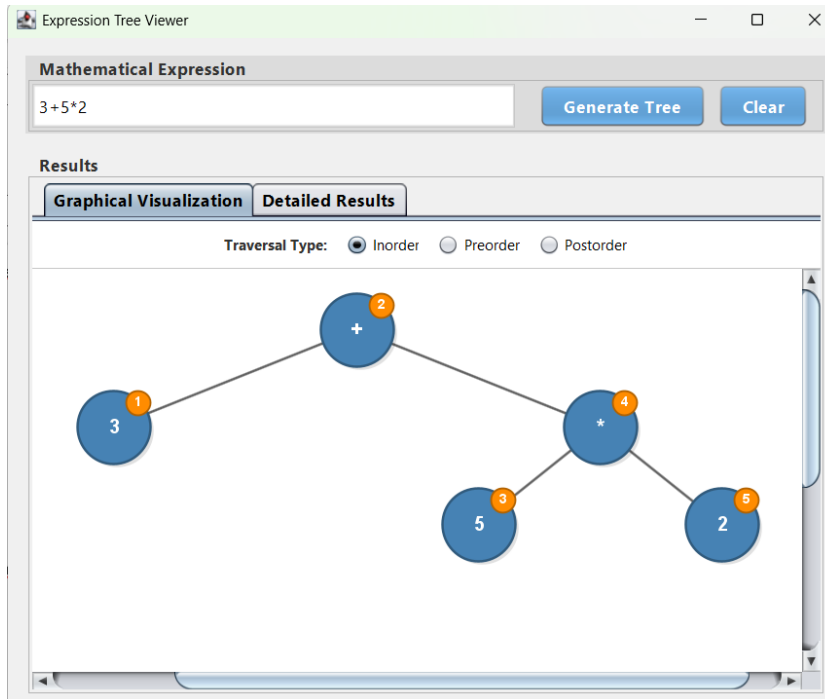
- Result: ✓ Correct - multiplication evaluated first

- Evaluation: 13

- Inorder: `3 + 5 \* 2`

- Preorder: `+ 3 \* 5 2`

- Postorder: `3 5 2 \* +`



Test Case 3: Parentheses Expression

- Input: (3+5)*2

- Expected Tree Structure:

```

      *
     /\
    + 2
   /\
  3 5
  
```

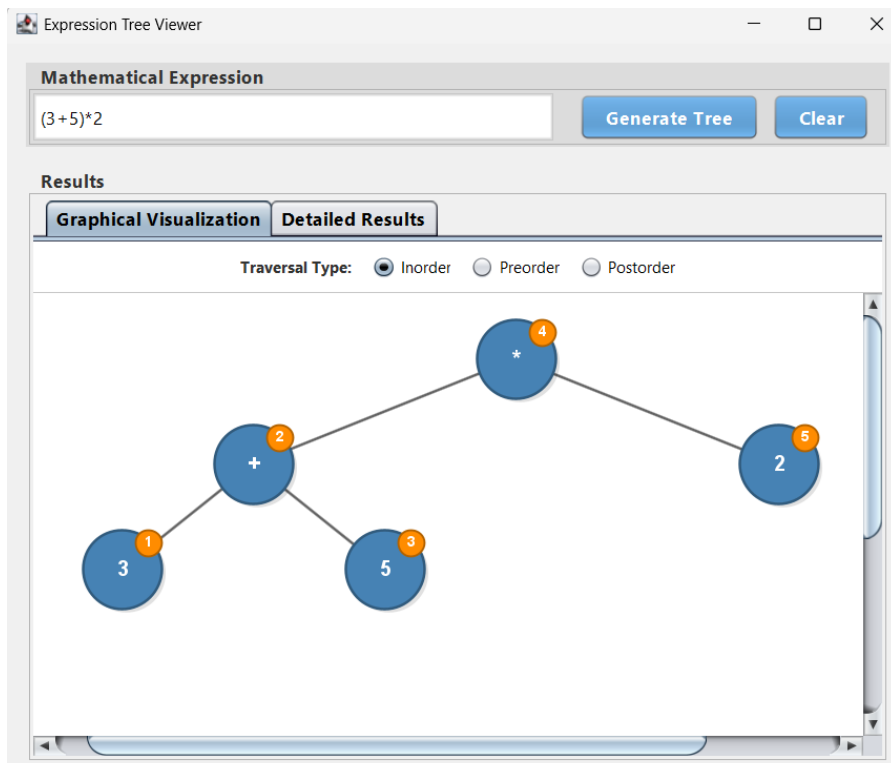

- **Result:** ✓ Correct - parentheses override precedence

- **Evaluation:** 16

- **Inorder:** `(3 + 5) \* 2`

- **Preorder:** `\* + 3 5 2`

- **Postorder:** `3 5 + 2 \*`



Mathematical Expression: Generate Tree Clear

Results

Graphical Visualization Detailed Results

Tree structure:

```

  2
 *
  5
 +
  3
  
```

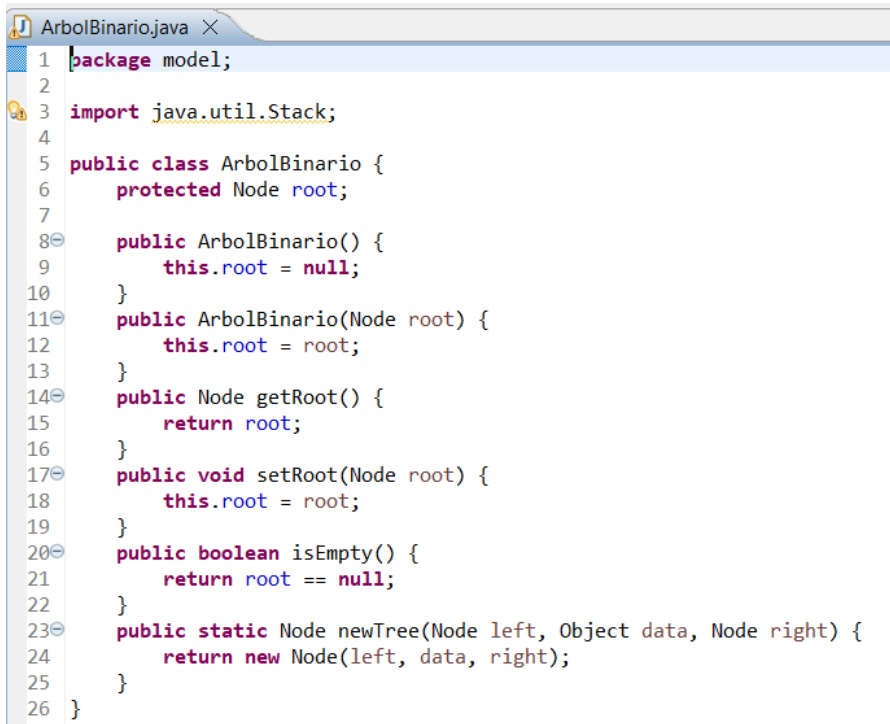
TREE TRAVERSALS

- Infix Traversal: `((3 + 5) * 2)`
- Prefix Traversal: `* + 3 5 2`
- Postfix Traversal: `3 5 + 2 *`

EVALUATION

Result: 16.0

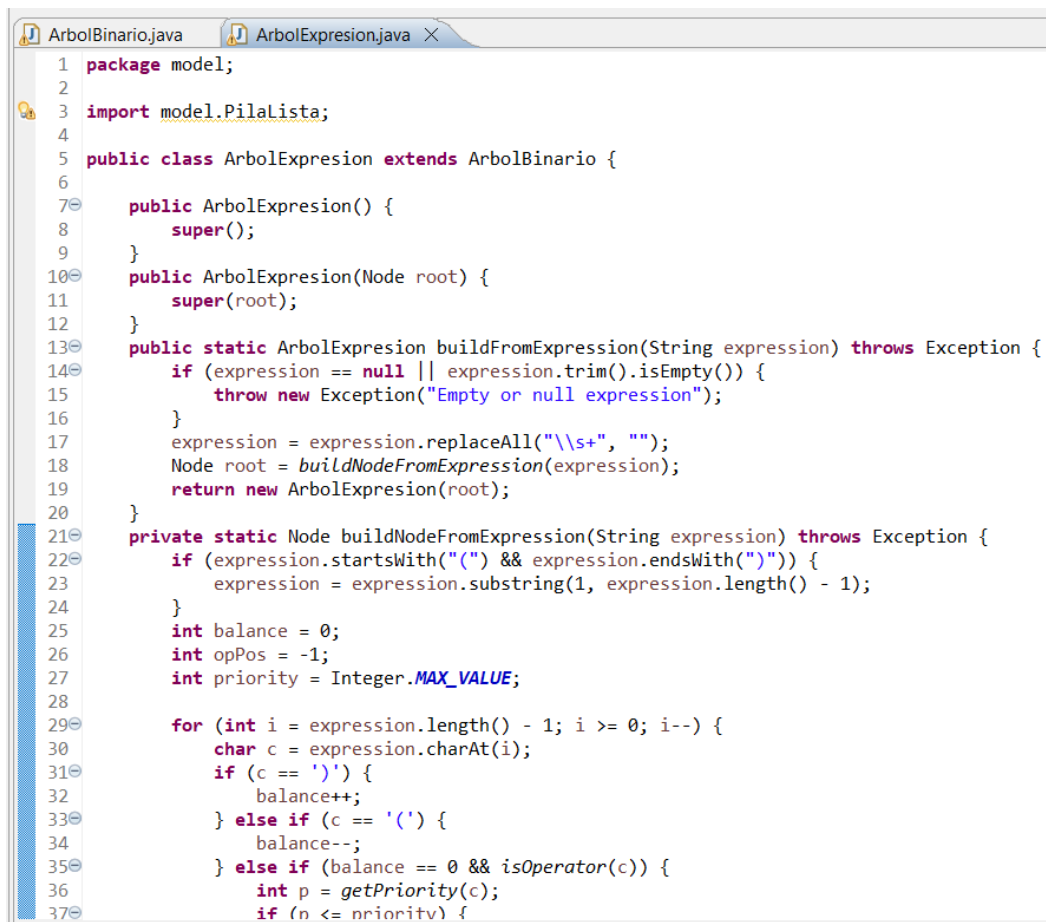
6. Graphics and photographs



```

1 package model;
2
3 import java.util.Stack;
4
5 public class ArbolBinario {
6     protected Node root;
7
8     public ArbolBinario() {
9         this.root = null;
10    }
11    public ArbolBinario(Node root) {
12        this.root = root;
13    }
14    public Node getRoot() {
15        return root;
16    }
17    public void setRoot(Node root) {
18        this.root = root;
19    }
20    public boolean isEmpty() {
21        return root == null;
22    }
23    public static Node newTree(Node left, Object data, Node right) {
24        return new Node(left, data, right);
25    }
26 }
  
```

Illustration 3 Binary tree template class



```

1 package model;
2
3 import model.PilaLista;
4
5 public class ArbolExpresion extends ArbolBinario {
6
7     public ArbolExpresion() {
8         super();
9     }
10    public ArbolExpresion(Node root) {
11        super(root);
12    }
13    public static ArbolExpresion buildFromExpression(String expression) throws Exception {
14        if (expression == null || expression.trim().isEmpty()) {
15            throw new Exception("Empty or null expression");
16        }
17        expression = expression.replaceAll("\\s+", "");
18        Node root = buildNodeFromExpression(expression);
19        return new ArbolExpresion(root);
20    }
21    private static Node buildNodeFromExpression(String expression) throws Exception {
22        if (expression.startsWith("(") && expression.endsWith(")")) {
23            expression = expression.substring(1, expression.length() - 1);
24        }
25        int balance = 0;
26        int opPos = -1;
27        int priority = Integer.MAX_VALUE;
28
29        for (int i = expression.length() - 1; i >= 0; i--) {
30            char c = expression.charAt(i);
31            if (c == ')') {
32                balance++;
33            } else if (c == '(') {
34                balance--;
35            } else if (balance == 0 && isOperator(c)) {
36                int p = getPriority(c);
37                if (p <= priority) {
  
```

Illustration 4 class containing tree operations

```

ArbolBinario.java  ArbolExpresion.java X
40      }
41    }
42  }
43
44  if (opPos != -1) {
45    String left = expression.substring(0, opPos);
46    String right = expression.substring(opPos + 1);
47    char op = expression.charAt(opPos);
48    return new Node(buildNodeFromExpression(left), String.valueOf(op), buildNodeFromExpression(right));
49  } else {
50    if (!containsOperator(expression)) {
51      return new Node(null, expression, null);
52    } else {
53      throw new Exception("Malformed expression: " + expression);
54    }
55  }
56
57
58  private static boolean isOperator(char c) {
59    return c == '+' || c == '-' || c == '*' || c == '/' || c == '^';
60  }
61
62  private static int getPriority(char op) {
63    switch (op) {
64      case '^':
65        return 3;
66      case '*':
67      case '/':
68        return 2;
69      case '+':
70      case '-':
71        return 1;
72      default:
73        return 0;
74    }
75  }
76

```

Illustration 5 Method for identifying operators

```

127 private String toInfixRecursive(Node node) {
128     if (node == null) return "";
129     if (node.leftSubtree() == null && node.rightSubtree() == null) {
130         return node.nodeValue().toString();
131     }
132     String left = toInfixRecursive(node.leftSubtree());
133     String right = toInfixRecursive(node.rightSubtree());
134     return "(" + left + " " + node.nodeValue() + " " + right + ")";
135 }
136

```

Illustration 6 Method for infix

```

141 private String toPrefixRecursive(Node node) {
142     if (node == null) return "";
143     String op = node.nodeValue().toString();
144     String left = toPrefixRecursive(node.leftSubtree());
145     String right = toPrefixRecursive(node.rightSubtree());
146     return op + " " + left + right;
147 }
148

```

Illustration 7 Method for prefix

```
153 private String toPostfixRecursive(Node node) {  
154     if (node == null) return "";  
155     String left = toPostfixRecursive(node.leftSubtree());  
156     String right = toPostfixRecursive(node.rightSubtree());  
157     String op = node.nodeValue().toString();  
158     return left + right + op + " ";  
159 }  
160
```

Illustration 8 Postfix method

7. CONCLUSIONES:

This laboratory successfully demonstrated the practical application of binary tree data structures in solving real-world problems. The combination of solid algorithmic implementation with an intuitive graphical interface creates an effective educational tool for understanding tree traversals and expression evaluation. The MVC architecture ensures the codebase is maintainable and extensible for future enhancements.

8. Recommendations

1. Practice with various expression types to understand operator precedence
2. Experiment with the three traversal types to see how they produce different notations
3. Study the recursive algorithms to understand tree processing patterns

9. BIBLIOGRAPHY:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms** (3rd ed.). MIT Press.
2. Weiss, M. A. (2011). *Data Structures and Algorithm Analysis in Java** (3rd ed.). Pearson.
3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java** (6th ed.). Wiley.
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms** (4th ed.). Addison-Wesley.
5. Horstmann, C. S. (2019). *Core Java Volume I – Fundamentals** (11th ed.). Prentice Hall.
6. Oracle. (2024). *Java Platform, Standard Edition Documentation**. Retrieved from <https://docs.oracle.com/en/java/javase/>
7. Oracle. (2024). *Java Swing Tutorial**. Retrieved from <https://docs.oracle.com/javase/tutorial/uiswing/>
8. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley.

10. LINK GITHUB:

[angeldev7/P2Lab1RodriguezAngel-estructuraDatos](https://github.com/angeldev7/P2Lab1RodriguezAngel-estructuraDatos)